Knowledge-Guided Selective Conformal Decision Support for Cross-Project Vulnerability Detection under Project Shift

Yanbo Bian[a,*], Zhihai Wang[b]

[a] *Computer Science and Technology Program, Weihai International College, Beijing Jiaotong University, Weihai 264401, China*
[b] *School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China*

Abstract

Problem: Cross-project vulnerability detection under project shift needs a decision rule that turns uncertain predictions into automatic labels or review actions under asymmetric security budgets. **Approach:** We present VulTriage, a knowledge-guided selective conformal decision-support layer whose explicit policy knowledge encodes class-specific risk budgets, evidence-sufficiency checks, a severe-violation threshold, and fail-closed action semantics. **Methods:** In a frozen, prediction-sealed evaluation, PrimeVul is used for retrospective gate development and 24 DiverseVul projects for external evaluation. We compare hashing-SGD with a frozen CodeBERT encoder. **Results:** At $(\alpha_{\mathrm{v}}, \alpha_{\mathrm{s}}) = (0.10, 0.20)$, weighting without the knowledge gate lowers hashing-SGD's project-paired maximum relative budget violation by 0.816 (95% interval [0.318, 1.722]; 24/24 projects) but lowers singleton coverage by 15.9 percentage points; it gives no overall risk-alignment improvement for CodeBERT. The frozen support gate has AUROC 0.8601 on external hashing, 0.4286 on external CodeBERT, and 0.35 in development. **Implications:** The evidence supports detector-conditional, auditable partial automation and exposes review cost; it does not establish a target-risk guarantee or detector-general validity.

*Corresponding author: Yanbo Bian.
Email addresses: 24722081@bjtu.edu.cn (Yanbo Bian), zhhwang@bjtu.edu.cn (Zhihai Wang)

Keywords: knowledge-guided decision support, vulnerability detection, conformal prediction, selective classification, dataset shift

1. Introduction

A vulnerability detector deployed on an unseen software project must answer two different questions. The first is predictive: which functions look vulnerable? The second is decisional: which of those predictions can be released automatically and which should be sent to review under an asymmetric security budget? A missed vulnerable function and an unnecessary alert consume different resources. A confidence score by itself does not reveal whether calibration evidence remains relevant to a new project, nor whether a requested risk budget is supported by enough effective observations. An intelligent decision-support system therefore needs explicit, auditable knowledge that connects uncertainty evidence to an action policy.

Public vulnerability benchmarks have made the first question more realistic. PrimeVul reports the effects of duplicate functions, label quality, imbalance, and evaluation design on code-language-model results [1], while DiverseVul broadens project and weakness coverage for deep vulnerability detection [2]. Cross-project methods address representation and distribution mismatch with transfer learning, domain adaptation, graph representations, and zero-shot mechanisms [3–8]. These methods improve or adapt a detector, but a deployment still needs an explicit policy for returning a label, deferring to review, and reporting the cost of that refusal.

Calibration and selective prediction provide useful ingredients. CodeBERT is a standard pretrained representation for source code [9], and LineVul and data-flow-aware models show how detector architecture can be improved [10, 11]. Temperature scaling is a canonical calibration baseline [12], while selective prediction formalizes the risk–coverage trade-off [13]. Code-model calibration can degrade under shift [14], and Prom provides a deployment-time conformal assessment for code-analysis tasks [15]. Recent work also frames code prediction as an answer-or-defer decision [16]; a KBS study similarly connects uncertainty estimation under dataset shift to malware-classification decisions [17]. These lines of work leave open a decision-support question: how should a project-level security tool expose uncertainty, refusal, and review cost when vulnerable and safe errors have different consequences?

We study VulTriage, a post-hoc decision layer that can wrap a fitted detector while being evaluated separately for each detector. It constructs

class-conditional prediction sets using estimated source-to-target relevance weights, then applies a frozen support gate based only on unlabeled target code and source calibration diagnostics. The learned detector supplies uncertainty evidence; an explicit knowledge layer supplies risk budgets, evidence constraints, and action semantics. A singleton set is an automatic label; a doubleton is review. The gate is not treated as a risk certificate. Instead, we test whether its label-free pass signal predicts severe raw violation after labels are released, separately for each detector. This framing makes the project, rather than an individual function or random row, the unit of deployment and inference.

The study asks:

- **RQ1:** Can a knowledge-constrained support policy provide auditable deploy-or-review decisions on unseen projects, and what automation cost does refusal impose?
- **RQ2:** Does estimated target weighting reduce class-asymmetric risk exceedance relative to unweighted sets, and does that effect replicate across detector types?
- **RQ3:** How do calibration size, duplicate control, and computation affect the operational validity of the decision layer?

The contributions are fourfold:

1. an explicit knowledge layer that separates learned uncertainty evidence from class-specific risk budgets, evidence-sufficiency checks, a severe-risk threshold, and fail-closed action semantics;
2. a frozen, prediction-sealed external-evaluation protocol combining a PrimeVul retrospective development domain, a DiverseVul external domain, two detectors, five seed addresses, a 3×3 asymmetric risk grid, and 25 recorded methods;
3. a detector-conditional risk-alignment result: on hashing-SGD, target weighting decreases the worst relative budget violation on all 24 projects at the primary operating point, with a quantified 15.9 percentage-point singleton-coverage cost; and
4. an external validity and reproducibility analysis showing that support discrimination does not transfer to CodeBERT, while separating support, singleton coverage, and label-revealed attainment in released summaries,

sensitivity analyses, figure source data, validation manifests, and a public builder that excludes raw code, labels, embeddings, private indexes, and per-function decision archives.

2. Related work

2.1. Cross-project vulnerability detection and decision support

Representation transfer for vulnerable-function discovery predates current code-language models [3]. CD-VulD combines deep domain adaptation with cross-domain vulnerability discovery [4]. CPVD uses graph attention and domain adaptation [5], DAM2P introduces a max-margin domain-adaptation objective for imbalanced data [6], and CSVD-TF uses TrAdaBoost to fuse expert and semantic metrics [7]. ZSVulD studies a zero-shot setting with pretrained representations and collaborative classifiers [8]. These systems change the detector or its training objective. VulTriage is complementary: it takes a fitted detector and exposes a decision layer that can refuse unsupported automation.

Software-engineering studies also emphasize the operational side of vulnerability prediction. A systematic mapping study organizes the evidence base and its evaluation practices [18]; comparative work shows that vulnerability predictors can behave differently across models and projects [19]; and a practical framework separates prediction, categorization, and localization rather than treating a single classifier score as the complete workflow [20]. These studies motivate a project-level decision-support layer that makes refusal cost and class-asymmetric risk explicit. VulTriage does not claim to replace those detectors or to provide a new vulnerability representation.

2.2. Knowledge-based uncertainty and security decision support

Recent contributions in *Knowledge-Based Systems* illustrate the target perspective. Yumlembam et al. connect uncertainty estimation with malware-classification decisions under dataset shift [17], while Wijekoon and Wiratunga evaluate user-facing counterfactual support for code vulnerability detection and correction [21]. Other KBS studies focus on improving the detector or the vulnerability score itself, including function fingerprints and code differences [22] and lightweight vulnerability-score prediction [23]. Our contribution is complementary: the learned detector remains fixed, and the knowledge component is an explicit, inspectable policy that maps uncertainty evidence to automatic or review actions. We therefore evaluate whether this policy

transfers across detector–shift pairs rather than presenting it as a new ontology, expert knowledge graph, or detector state of the art.

Modern code detectors provide important external-validity baselines. CodeBERT supplies a pretrained code representation [9], LineVul adapts transformers to line-level vulnerability prediction [10], and DeepDFA injects data-flow structure for efficient detection [11]. Our CodeBERT track is deliberately a frozen encoder plus a source-train-only standardized logistic head, not a claim of fine-tuned state of the art. The contrast with hashing-SGD therefore tests whether support-aware weighting behaves consistently across a low-cost lexical detector and a semantic representation. These studies primarily optimize or compare detectors; VulTriage addresses the complementary software-engineering task of deciding when detector output is ready for automatic release and when it must remain in a review queue.

2.3. Selective automation and conformal risk

Temperature scaling changes probability calibration but not the binary ranking [12]. Selective prediction makes retained coverage an explicit companion to risk [13], and code-model studies show that calibration remedies can behave differently under project or temporal shift [14, 24]. Prom uses local conformal experts, including LAC, TopK, APS, and RAPS, to identify deployment-time errors in code analysis [15]. We use a pre-specified binary, deterministic PROM-derived adapter as a strong comparison, while distinguishing it from the official implementation. The answer-or-defer perspective for code prediction is also established [16]; our focus is the security-specific separation of vulnerable and safe budgets and the validity of an unlabeled support signal.

Weighted conformal prediction can recover target marginal coverage under covariate shift when the likelihood ratio is known and conditional labels are invariant [25]. Conformal risk control generalizes the controlled loss [26], and nonexchangeable or high-probability variants make the shift assumptions explicit [27, 28]. Our density ratios are learned from a finite domain classifier and project shift may include concept shift. We therefore report empirical violation, support diagnostics, and refusal cost rather than claiming a distribution-free deployment guarantee.

3. Knowledge-guided selective decision support

3.1. Policy knowledge and action semantics

We separate the learned detector from the explicit policy knowledge used to decide whether its output may be automated. Let

$$\mathcal{K} = \{\boldsymbol{\alpha}, \mathcal{E}, \tau_{\text{sev}}, \mathcal{A}\}, \tag{1}$$

where $\boldsymbol{\alpha} = (\alpha_{\text{v}}, \alpha_{\text{s}})$ contains the vulnerable- and safe-class risk budgets, $\mathcal{E}$ contains admissibility checks for finite effective-sample-size, test-point-mass, domain-AUROC, and clipping diagnostics, $\tau_{\text{sev}} = 0.5$ is the severe-relative-violation threshold, and $\mathcal{A}$ defines the automatic-label and review actions. For a target project, the gate input z contains only unlabeled diagnostics. The knowledge controller $G_{\mathcal{K}}(z)$ passes only when the checks in $\mathcal{E}$ and the fixed severe-risk rule are satisfied.

In the frozen implementation, $\mathcal{E}$ is a serializable label-free evidence vector rather than a hand-written ontology. It records total and class-specific Kish effective sample sizes, test-point infinity mass, domain-classifier AUROC, and clipping fractions. The primary full gate uses these values as features of a frozen logistic severe-risk model; it requires the feature vector to be finite, but does not impose separate hand thresholds on each feature. The ESS-only and infinity-mass-only variants below are explicit hard-check ablations of this evidence policy.

Given a prediction set $\Gamma(x)$, the action policy is

$$\pi_{\mathcal{K}}(x, z, \Gamma) = \begin{cases} \textsf{auto-1}, & G_{\mathcal{K}}(z) = 1 \text{ and } \Gamma(x) = \{1\}, \\ \textsf{auto-0}, & G_{\mathcal{K}}(z) = 1 \text{ and } \Gamma(x) = \{0\}, \\ \textsf{review}, & \text{otherwise.} \end{cases} \tag{2}$$

This is a knowledge-guided controller rather than an expert-elicited ontology: its value is traceability. Each automatic action can be linked to a requested risk budget and a recorded evidence check, while a failed check closes the system toward review instead of forcing a label. The empirical study tests this policy and its learned weighting component separately.

3.2. Uncertainty sets and three-way actions

Let $Y \in \{0, 1\}$ denote safe and vulnerable, respectively, and let a fitted detector output class probabilities $\widehat{p}_y(x)$. For calibration example (x_i, y_i), the label-conditional nonconformity score is

Table 1: Serializable policy knowledge and its runtime interpretation. The rules are fixed before target labels are joined.

Rule	Knowledge item	Runtime interpretation	Audit record
K1	Class-specific risk budgets	Bind α_{v} and α_{s} to the two class-conditional prediction sets and their relative-violation scores.	Budget pair and class-wise outcomes are stored per project.
K2	Label-free support evidence	Record ESS, infinity mass, domain AUROC, and clipping fractions; the full gate accepts only a finite evidence vector.	Evidence features are sealed before target labels are joined.
K3	Severe-risk controller	Apply the PrimeVul leave-one-project-out-fitted logistic model; a probability below 0.5 is eligible to pass the full gate.	Gate probability, model hash, and threshold are recorded.
K4	Singleton eligibility	Require at least one singleton prediction set before any automatic action is allowed.	Pass/fail status and singleton coverage are reported separately.
K5	Fail-closed action semantics	A failed, missing, or nonfinite support check converts the whole project cell to review rather than forcing a label.	Review load is retained and label-revealed attainment is analyzed separately.

$$S_i = 1 - \widehat{p}_{y_i}(x_i). \tag{3}$$

The output $\Gamma(x) \subseteq \{0, 1\}$ has three operational interpretations. $\{0\}$ and $\{1\}$ are singleton automatic decisions; $\{0, 1\}$ is a review decision. Empty sets, which can arise in comparison methods, are also routed to review and are recorded separately. For class y, the empirical miscoverage is

$$R_y = \Pr\{y \notin \Gamma(X) \mid Y = y\}. \tag{4}$$

The user supplies $\alpha_1 = \alpha_{\mathrm{v}}$ and $\alpha_0 = \alpha_{\mathrm{s}}$. Singleton coverage is $\Pr(|\Gamma(X)| = 1)$ and review load is its complement.

3.3. Asymmetric weighted sets

For class y, let $\mathcal{I}_y$ be its calibration indices and let w_i be estimated target relevance weights. For a target point with weight $w(x)$, the implemented threshold is

$$q_y(x) = \inf \left\{ t : \sum_{i \in \mathcal{I}_y} w_i \mathbf{1}(S_i \le t) \ge (1 - \alpha_y) \left(\sum_{i \in \mathcal{I}_y} w_i + w(x) \right) \right\}. \quad (5)$$

If the calibration mass cannot reach the right-hand side, $q_y(x) = \infty$. The set contains class y when $1 - \widehat{p}_y(x) \le q_y(x)$. Unit weights recover unweighted Mondrian conformal prediction. The weighted construction is an empirical post-processing rule: its exact covariate-shift validity would require a correct likelihood ratio and invariant conditional labels [25].

3.4. Project-shift weights and support gate

For each held-out project, a source–target domain classifier is trained with five deterministic cross-fitting folds. Equal source and target sampling makes the cross-fitted target odds

$$\widehat{w}(x) = \frac{\widehat{P}(D = 1 \mid x)}{1 - \widehat{P}(D = 1 \mid x)} \quad (6)$$

an estimated relevance ratio. The primary weights are clipped to $[1/20, 20]$; clips 10 and 50 are sensitivity variants. The same label-free weights are shared by detector-head seeds.

The frozen support gate is fitted only on 12 PrimeVul project-disjoint pseudo-targets using leave-one-project-out cross-fitting. Its features are log total and class-specific Kish effective sample sizes (ESS), maximum and 99th-percentile test-point infinity mass, domain-classifier AUROC, and lower and upper clipping fractions. The gate predicts whether the raw weighted policy will have severe relative violation, defined as

$$\max \left(\frac{\max(0, R_1 - \alpha_{\mathrm{v}})}{\alpha_{\mathrm{v}}}, \frac{\max(0, R_0 - \alpha_{\mathrm{s}})}{\alpha_{\mathrm{s}}} \right) > 0.5. \quad (7)$$

The final controller $G_{\mathcal{K}}$ passes only when its fixed logistic score (interpreted as an estimated severe-violation probability, without post-hoc target calibration) is below $\tau_{\mathrm{sev}} = 0.5$, the evidence vector is finite, and at least

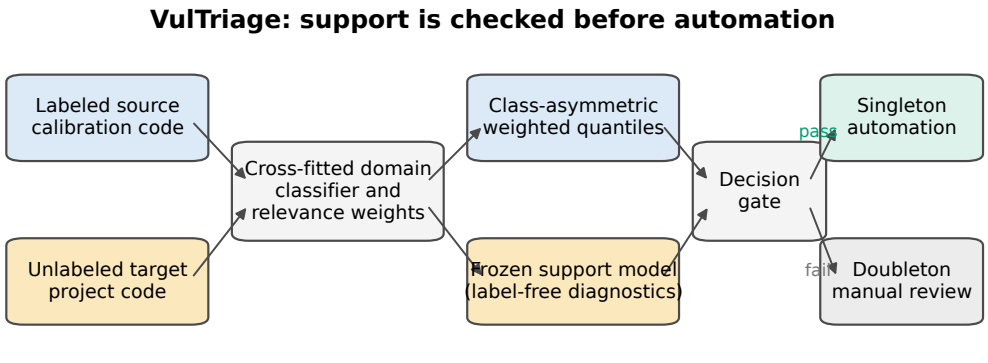


Figure 1: VulTriage uses labeled source calibration code and unlabeled target-project code to estimate relevance, construct class-asymmetric weighted sets, and apply a frozen support model before automation. A failed support check returns a review set rather than a forced label.

one singleton exists. Missing or nonfinite diagnostics fail closed. A failed controller replaces the entire target-project cell with review sets. This is a conservative knowledge rule, not evidence that the requested risks have been attained. The development fit contains 12 independent projects and nine correlated budget rows per project; the resulting 108-row AUROC is therefore descriptive rather than an independent-sample estimate.

4. Empirical evaluation of the decision-support system

4.1. Study protocol and analysis unit

The study follows a frozen, prediction-sealed external-evaluation protocol organized around the software-project unit of deployment. PrimeVul is used for retrospective development and gate fitting, whereas DiverseVul supplies the external project groups. Target labels are withheld until detector predictions and support diagnostics are sealed. The learned weighting component and the explicit knowledge controller $\mathcal{K}$ are evaluated as separate decision-support components. The primary comparison is estimated weighting versus unweighted Mondrian sets at $(\alpha_{\mathrm{v}}, \alpha_{\mathrm{s}}) = (0.10, 0.20)$ within each detector; the remaining budget cells, calibration fractions, duplicate audit, and efficiency measurements are sensitivity or descriptive analyses. Five seeds are technical

Table 2: DiverseVul external project groups. Target rows are exact-deduplicated functions; vulnerability prevalence is reported only descriptively after label release.

Project	Total	Vulnerable	Prev. (%)	Project	Total	Vulnerable	Prev. (%)
linux	32,930	3,032	9.21	openssl	4,352	565	12.98
tensorflow	2,677	438	16.36	php	3,592	353	9.83
qemu	2,634	338	12.83	imagemagick	783	300	38.31
qpdf	1,786	272	15.23	envoy	5,252	254	4.84
tcpdump	428	209	48.83	jasper	608	203	33.39
radare2	1,610	168	10.43	postgres	2,605	154	5.91
mysql-server	3,838	142	3.70	FreeRDP	980	141	14.39
libxml2	2,060	136	6.60	krb5	491	118	24.03
samba	1,961	117	5.97	vim	3,056	116	3.80
net	1,810	116	6.41	ffmpeg	983	115	11.70
mongo	2,627	112	4.26	ghostpdl	1,132	109	9.63
libtpms	490	109	22.24	librsvg	670	95	14.18

replicates nested within a project, so inference is performed on the 24 external projects rather than on individual functions or seed–project rows.

4.2. Datasets and project splits

The development domain is the original public PrimeVul release [1]. After deterministic canonicalization and exact deduplication, the source package contains 226,582 functions from 235,768 valid rows; 9,186 same-label duplicates are removed and no conflicting label keys are retained. For each external target, source rows are partitioned by salted commit-hash buckets into 70% detector training, 10% model validation, and 20% calibration. Project aliases are frozen before model output.

The external evaluation domain is the official public DiverseVul release [2]. We select 24 eligible project groups without inspecting model outcomes, require at least 200 total, 20 vulnerable, and 20 safe functions, and then exact-deduplicate across datasets. Eligibility uses target label counts only to exclude one-class or very small projects; it is a cohort-design criterion rather than an outcome-based selection, but it limits prevalence generalization. The resulting target cohort contains 79,355 functions. Labels remain in a separate vault until both detector prediction seals are verified. The primary cohort is unchanged by the near-duplicate sensitivity analysis.

4.3. Detectors and comparison methods

The lightweight detector hashes case-sensitive word unigrams and bigrams into 2^{18} dimensions, applies ℓ_2 normalization, and fits a class-balanced averaged-SGD logistic head for three epochs. Regularization is selected by source-validation PR-AUC. Seeds 13, 37, 73, 101, and 137 are five independently fitted technical seeds per project.

The semantic track uses `microsoft/codebert-base` at immutable revision `3b0952feddeffad0063f274080e3c23d75e7eb39`. Final hidden states are attention-mask mean pooled after removing special tokens. A source-train-only StandardScaler feeds a class-weighted L2 liblinear logistic head; $C \in \{0.01, 0.1, 1, 10\}$ is selected on source validation. CodeBERT is fitted once at seed 13 per project and the deterministic head is copied to the other four frozen seed addresses. This is a reproducibility address scheme, not five independent CodeBERT fits.

The evaluation records forced argmax, matched maximum-softmax and entropy, temperature-matched confidence, pooled and unweighted conformal sets, a binary PROM-derived adapter (LAC, TopK, APS, RAPS, and their union), estimated weighting with clips 10/20/50, ESS-only and infinity-mass-only gates, the full frozen gate, and matched full-gate derivatives. The PROM comparison is explicitly a deterministic binary adapter, not an official PROM reproduction [15].

4.4. Outcomes, sealing, and statistics

We evaluate nine pre-specified budget pairs: $\alpha_{\mathrm{v}} \in \{0.01, 0.05, 0.10\}$ and $\alpha_{\mathrm{s}} \in \{0.05, 0.10, 0.20\}$. Primary outcomes are vulnerable and safe miscoverage, positive class-specific violation, maximum relative violation, singleton coverage, and review load. Detector performance includes PR-AUC, F1, Matthews correlation coefficient (MCC), Brier score, expected calibration error, area under the risk–coverage curve, and error-detection AUROC. The latter ranks forced 0.5-threshold mistakes by $1 - \max_y \widehat{p}_y$; it is not a vulnerable-classification ROC-AUC.

Prediction files are hashed and sealed before target labels are joined. The final evaluation contains 54,000 fold–seed metric rows, 10,800 project–seed averages, 450 paired comparisons, 14,400 calibration-size rows, and 6,480 near-duplicate sensitivity rows. Five seeds are averaged within each project before inference; the independent unit is the 24-project target group. Project-bootstrap intervals use 10,000 resamples. Exact Wilcoxon tests are

Table 3: Base detector performance across 24 external projects. Values are project medians with the interquartile range in parentheses; confidence intervals are 10,000-project bootstrap intervals. Error-detection AUROC is not vulnerable-class ROC-AUC.

Metric	Hashing-SGD	CodeBERT
PR-AUC	0.188 (0.149–0.295)	0.193 (0.141–0.278)
F1	0.268 (0.197–0.343)	0.271 (0.186–0.366)
MCC	0.162 (0.113–0.198)	0.183 (0.143–0.233)
Brier (lower)	0.194 (0.151–0.240)	0.203 (0.173–0.236)
Error-detection AUROC	0.677 (0.625–0.733)	0.718 (0.654–0.762)

emitted only when at least ten nonzero paired differences remain, with Holm adjustment within each detector, budget, and outcome family.

5. Evaluation results

5.1. Base detectors are comparable, not interchangeable

Table 3 summarizes the two detector tracks at the primary operating point. CodeBERT has a modestly higher median PR-AUC, F1, and MCC, but project-bootstrap intervals for PR-AUC, F1, and MCC overlap the hashing track. Its error-detection AUROC is higher, whereas its Brier score is also higher (worse). The decision-layer results below should therefore be interpreted as detector-conditional rather than as a model-superiority claim.

5.2. Knowledge-gate support is detector-conditional

Figure 2 combines the negative development result with the external evaluation. The frozen gate has PrimeVul leave-one-project-out AUROC 0.35 and AUPRC 0.2452 over 108 development cells. On DiverseVul, it has AUROC 0.8601 and AUPRC 0.8173 for hashing-SGD; pass projects have a median raw maximum-relative-violation difference of -0.5473 versus fail projects (95% interval $[-1.0057, -0.2244]$). For CodeBERT the AUROC is 0.4286. Its AUPRC 0.8733 is close to the severe-event prevalence of 0.875, and the pass–fail interval $[-2.4498, 0.7270]$ crosses zero. The gate therefore has external discrimination for hashing-SGD, not a detector-general support signal.

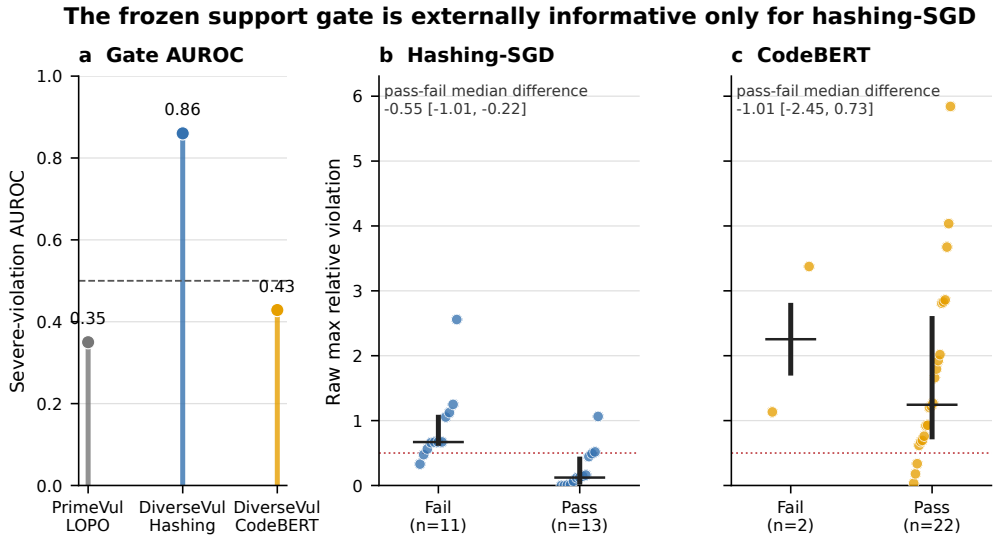


Figure 2: Detector-conditional validity of the frozen support gate. (a) Severe-violation AUROC includes the negative PrimeVul development result and the two DiverseVul external detector tracks; the dashed line is chance. (b,c) Raw maximum-relative violation by all-five-seed gate status at the primary operating point. Points are projects; black marks show the interquartile range and median; the red dotted line is the severe-violation threshold. Pass–fail differences are descriptive associations, not causal effects.

5.3. Evidence weighting improves risk alignment for hashing-SGD

At $(\alpha_v, \alpha_s) = (0.10, 0.20)$, estimated weighting with clip 20 reduces hashing-SGD maximum relative violation by a median 0.8161 compared with unweighted Mondrian sets (95% project-bootstrap interval for the signed difference $[-1.7219, -0.3174]$; 24/24 project directions improve; Holm-adjusted exact Wilcoxon $p = 5.96 \times 10^{-7}$). Vulnerable-class violation decreases by 0.0853 (23 wins, one tie; adjusted $p = 1.19 \times 10^{-6}$). Safe-class violation has no stable change (median difference 0.0000; adjusted $p = 0.9748$). The corresponding 0.1587 reduction in singleton coverage occurs in all 24 projects (adjusted $p = 5.96 \times 10^{-7}$), so the risk alignment is not free automation.

The detector contrast is decisive. CodeBERT's maximum-relative-violation difference is -0.0052 with interval $[-0.2837, 0.5505]$ and 12 wins versus 12 losses (adjusted $p = 1$). Vulnerable violation is directionally worse by 0.0298, while safe violation improves by 0.0075; singleton-coverage change is -0.0235 with an interval crossing zero. Across all nine budget cells, the weighted hashing policy improves, ties, or worsens 202, 6, or 8 of 216

Table 4: Clip-20 evidence-rule ablation at $(\alpha_v, \alpha_s) = (0.10, 0.20)$. For gate variants, medians are computed over support-pass projects; for the no-gate row, all 24 projects are included and support-pass is not applicable.

Policy	Hashing-SGD			CodeBERT		
	Pass / 24	Max rel. violation	Cov.	Pass / 24	Max rel. violation	Cov.
Weighted, no gate	n/a	0.483	0.524	n/a	1.244	0.749
ESS-only gate	24	0.483	0.524	24	1.244	0.749
Infinity-mass-only gate	20	0.483	0.554	24	1.244	0.749
Full frozen gate	13	0.122	0.509	22	1.244	0.749

project–budget cells; CodeBERT gives 97, 32, or 87. These cell counts are descriptive because the nine cells within a project are correlated.

Table 4 isolates the evidence rules at the selected clip of 20. The ESS-only variant passes every project and is numerically identical to the no-gate weighted policy for both detectors in this cohort. The infinity-mass-only variant refuses four hashing projects but does not improve the supported median risk relative to the no-gate policy. The full gate is the only variant that uses the frozen severe-risk model and materially reduces the supported hashing median, at the cost of additional refusal. Its supported CodeBERT median remains unchanged, which is consistent with the detector-conditional gate result above.

5.4. Knowledge constraints expose automation cost

At the primary operating point, the full gate supports 13/24 hashing projects and 22/24 CodeBERT projects. Among supported hashing projects, median singleton coverage is 0.509 and maximum relative violation is 0.122; only 2/13 supported projects attain both class budgets after labels are released. For CodeBERT, supported median coverage is 0.749, maximum relative violation is 1.244, and supported attainment is 0/22. The apparent overall full-gate attainment of 13/24 for hashing and 2/24 for CodeBERT is therefore dominated by gate-fail projects that are sent entirely to review and consequently have zero automatic violation. We do not count those mechanical zeros as evidence of predictive risk control.

Figure 4 shows the corresponding supported-project coverage. The hashing median is 50.9% and the CodeBERT median is 74.9%. The difference is operationally important: a system that refuses more often can look safer

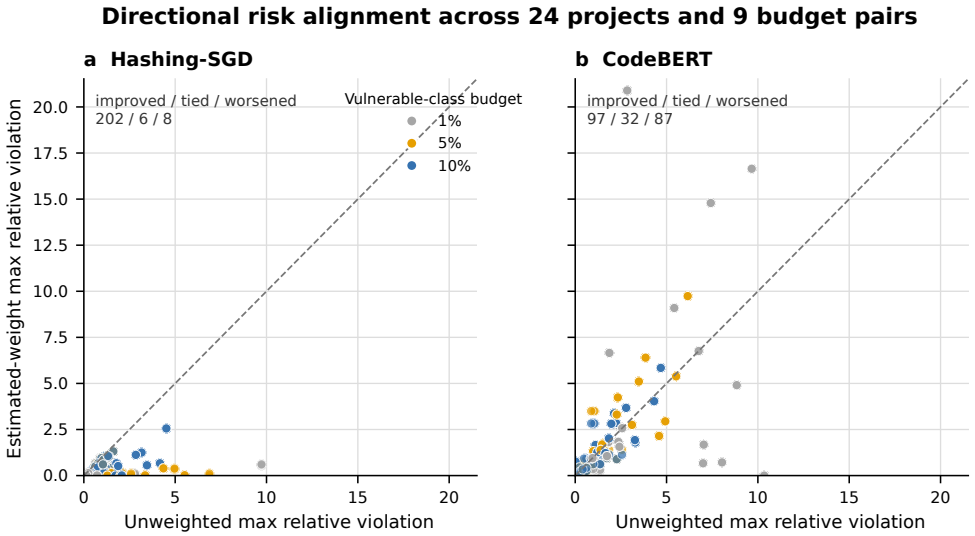


Figure 3: Directional risk alignment over the 24 projects and nine frozen budget pairs. Each point compares raw estimated-weight sets with unweighted Mondrian sets; points below the diagonal improve maximum relative violation. Colors encode the vulnerable-class budget. The improved/tied/worsened counts are descriptive cell counts, not independent sample sizes.

while increasing analyst workload. PROM-derived APS and RAPS provide another reminder of this trade-off, with about 24–26% coverage and higher descriptive attainment; TopK and the PROM union reject every function. These baselines are retained as controls, not presented as practical dominance.

5.5. Sensitivity and resource observations

The calibration-size sensitivity preserves the central detector contrast (Fig. 5). For hashing-SGD, weighted versus unweighted median maximum relative violation is 0.099 versus 1.271 at 25% calibration, and 0.483 versus 1.242 at 100%; weighted coverage rises from 35.5% to 52.4% while remaining below unweighted coverage of 69.4% to 69.7%. Full-gate pass rates rise from 0 at 25% to 0.542 at 100%, but the 25% full-gate zero attainment is entirely review-only. CodeBERT has no consistent weighting advantage: its weighted maximum relative violation is 1.290, 1.296, 1.295, and 1.244 across the four fractions, versus 1.270, 1.224, 1.219, and 1.238 unweighted.

The near-duplicate audit generated 866,094 approximate-LSH candidates and exactly verified every candidate. It flagged 24,606 pairs at lexical-token Jaccard at least 0.90, affecting 15,026 of 79,355 target rows (18.94%) and

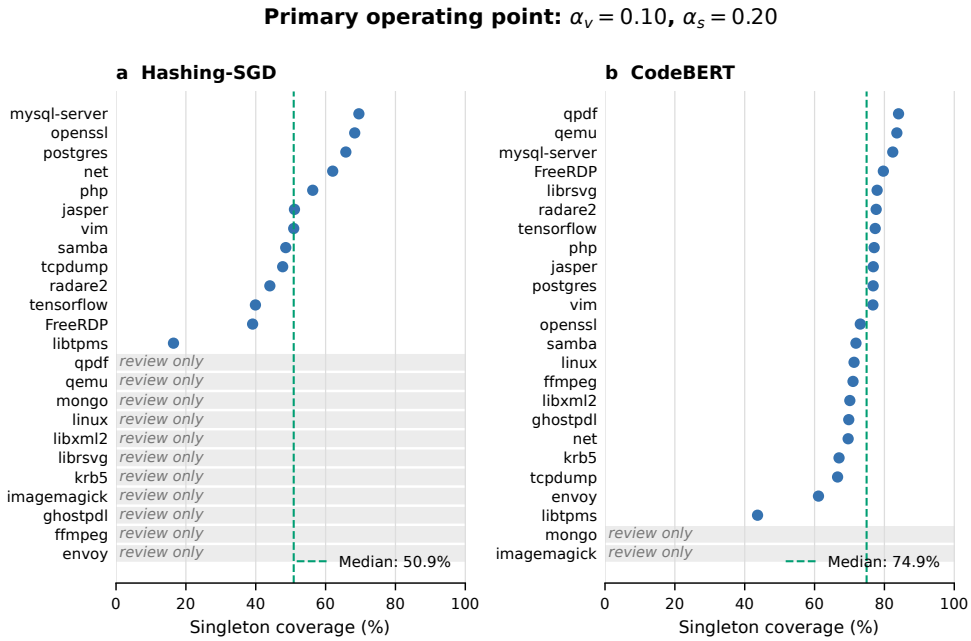


Figure 4: Singleton coverage at $(\alpha_{\mathrm{v}}, \alpha_{\mathrm{s}}) = (0.10, 0.20)$. Each point is a supported project; gray rows are gate-fail projects routed entirely to review. The dashed line is the median coverage among supported projects. Coverage is not target-risk attainment.

retaining 64,329 rows. Recomputing sealed decisions on the clean sensitivity cohort preserves the hashing direction: weighted maximum relative violation changes from 0.483 to 0.434 and coverage from 0.524 to 0.528, whereas CodeBERT changes from 1.244 to 1.799 and from 0.749 to 0.769. This is a sensitivity re-evaluation, not a refit.

The efficiency audit is observational. Hashing executes 120 independent head fits with a median 1.83 s fit duration, whereas CodeBERT executes 24 actual deterministic head fits with a median 1,584 s duration after 1,522 s of embedding extraction. CodeBERT runs in four concurrent parts; hashing is recorded as a sequential pipeline. Because hardware, batching, and parallelization differ, these numbers do not support a speedup, energy, or architecture-efficiency claim. They do support reporting the resource cost of the semantic track and the seed-reuse policy.

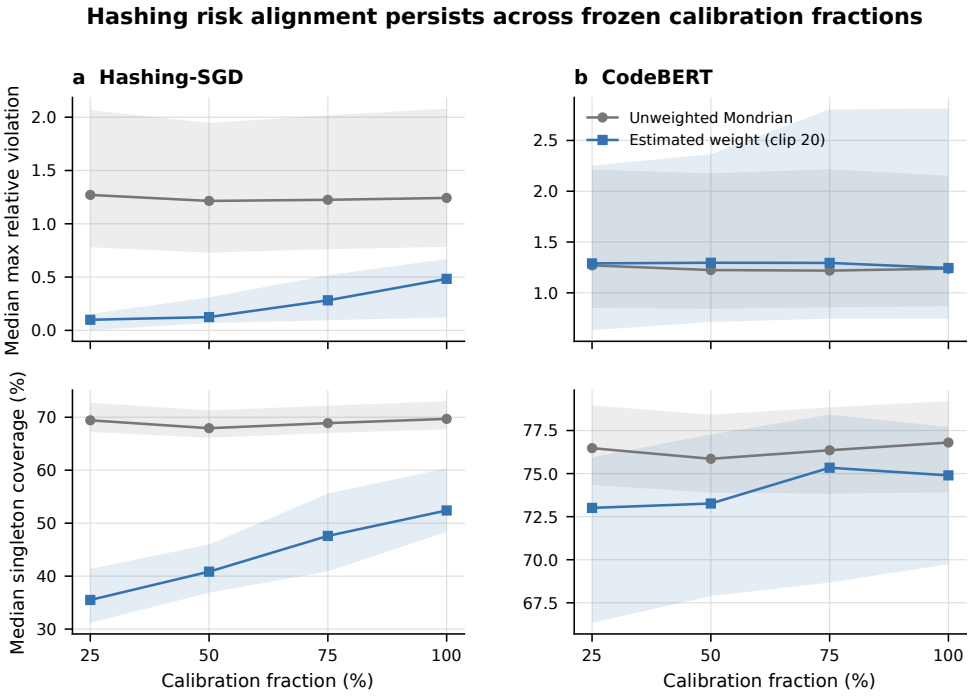


Figure 5: Calibration-size sensitivity at the primary budget. Lines show project medians and ribbons show the interquartile range across 24 projects. Weighted hashing retains lower maximum relative violation at every fraction but lower singleton coverage; CodeBERT has no consistent risk-alignment advantage.

6. Discussion

6.1. Answers to the research questions

For RQ1, the knowledge-constrained support policy produces partial, auditable automation but does not guarantee that the requested budgets are attained. At the primary operating point, the full gate supports 13 of 24 hashing projects and 22 of 24 CodeBERT projects. Only 2 of the 13 supported hashing projects attain both class budgets after labels are released, and none of the 22 supported CodeBERT projects does so. The controller therefore creates a review boundary that can be audited before labels are available; it should not be reported as a success merely because gate-fail projects produce zero automatic violations. The knowledge contribution is procedural and inspectable: it makes the risk budget, evidence checks, and fail-closed action explicit, while leaving the empirical validity of those rules open to detector-specific testing.

For RQ2, estimated weighting reduces risk exceedance for hashing-SGD and not for CodeBERT. The primary hashing comparison improves on all 24 external projects, whereas the CodeBERT comparison has 12 wins and 12 losses with an interval that crosses zero. The answer is therefore detector-conditional: the same post-hoc policy cannot be assumed to transfer across detector families.

For RQ3, the operational trade-offs are visible in the sensitivity and efficiency analyses. More calibration data increases gate pass rates, duplicate removal changes the semantic track more than the hashing track, and the semantic representation incurs substantially higher measured resource cost under the recorded execution regime. These observations define deployment conditions and costs rather than universal performance claims.

6.2. Interpretation of the detector-conditional result

The strongest conclusion is a decision-layer result for a lightweight detector. When relevance weighting is applied without the gate, the hashing track reduces class-asymmetric risk exceedance across projects and budget cells. The effect survives a smaller calibration cohort and a restrictive near-duplicate sensitivity cohort. A deployer can therefore use the method as an auditable risk–coverage surface: choose budgets, inspect label-free support, and decide how much review capacity is acceptable.

This result is deliberately narrower than model superiority. CodeBERT has similar base detection metrics and higher error-detection AUROC, yet the same weighting rule does not improve overall risk alignment. A support model trained on PrimeVul also performs poorly in leave-one-project-out development evaluation. The external hashing result is encouraging because it is obtained on a separate dataset, but the detector contrast means that support is a property of the detector–shift pair, not a universal certificate.

6.3. Why deployment metrics must be separated

Support qualification is a pre-label diagnostic. Singleton coverage measures how much automation remains after the policy. Attainment measures whether the observed target violations satisfy both budgets after labels are released. These quantities can move in opposite directions. In particular, a failed gate sends all functions to review, producing zero automatic violation and zero coverage. Reporting only overall attainment would reward refusal and conceal the operational cost. Our tables and evidence package report the supported cohort separately and keep the negative development gate result visible.

6.4. Implications for knowledge-based decision support

The practical output of VulTriage is a traceable knowledge-based decision record rather than a replacement vulnerability detector. For each project, a deployment can store the requested class budgets, the label-free support diagnostics, the controller decision, the number of singleton and review outputs, and the later label-revealed violations. This record lets a security team audit why a project was automated or deferred and compare review demand across projects. It also makes refusal an explicit resource decision: a stricter controller can reduce automatic exposure while increasing analyst workload.

The study suggests two implementation safeguards for knowledge-based systems. First, support qualification should be calibrated and monitored separately for each detector–shift pair; a controller trained with one detector must not be silently reused for another. Second, dashboards should show support, coverage, and attainment as separate quantities, with the active policy knowledge and its threshold values recorded alongside them. The public artifact provides the corresponding aggregates and validation manifests, but a production adoption would still require local calibration, independent detector fits, expert review of the policy rules, and a human productivity assessment.

6.5. Threats to validity

Construct validity. Singleton coverage is a proxy for automation volume, not analyst time saved. We conduct no human productivity study. A safe singleton is not a security certification, and empirical budget attainment is not a finite-sample guarantee.

Knowledge specification validity. The policy knowledge in $\mathcal{K}$ is authored from the study's security-risk and evidence requirements; it is not elicited from a panel of practitioners and is not a general-purpose knowledge graph. The threshold values and fail-closed semantics should therefore be treated as an auditable prototype policy that requires local governance and sensitivity analysis before operational use.

Internal validity. Prediction seals prevent target-label access during model output, and exact deduplication plus alias maps are frozen. Public benchmark labels can still be incomplete or noisy. CodeBERT seed addresses are deterministic copies of one head per project, so they do not quantify independent encoder or optimization variation. The gate pass–fail comparison is associational, not causal.

Statistical conclusion validity. The independent sample is 24 projects, not 120 seed–project cells or 216 independent budget cells. The support-gate development fit has 12 independent PrimeVul projects but 108 project–budget rows, so its AUROC is descriptive and not a 108-sample generalization estimate. Bootstrap intervals quantify project heterogeneity. Holm correction is restricted to the frozen comparator family within each detector, budget, and outcome. Sensitivity fractions and near-duplicate rows are descriptive re-evaluations.

External validity. The study covers public C/C++ functions and two detector tracks. It does not establish behavior for fine-tuned CodeBERT, LineVul, data-flow models, other languages, private repositories, or concept shifts with changing vulnerability mechanisms. The hashing result requires replication with independently fitted semantic detectors and additional external projects.

6.6. Responsible-use boundary

The study uses public source-code records and vulnerability labels for defensive classification. It does not generate exploits or access live systems, and it does not involve human participants. The method is evaluated as a decision-support layer for vulnerability triage, not as a security certification or an autonomous remediation system.

7. Conclusion

VulTriage turns cross-project vulnerability uncertainty into an explicit, knowledge-guided deploy-or-review interface. Under a frozen, prediction-sealed two-dataset evaluation, estimated weighting improves worst-class budget alignment across all 24 DiverseVul projects for hashing-SGD at the primary operating point, with a clear 15.9 percentage-point automation cost. The result survives calibration-size and near-duplicate sensitivity, while the support controller shows external discrimination only for the hashing detector and not for CodeBERT. The evidence supports an auditable prototype of selective decision support, not universal target-risk guarantees, detector-general support validity, or a claim that the authored policy knowledge transfers without local governance.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

PrimeVul and DiverseVul are public defensive-use datasets released by their respective authors [1, 2]. The public repository at `https://github.com/bianyanbo44-afk/vultriage-fcs` contains the frozen protocol, label-free source and target metadata, evaluation and analysis code, summary tables, figure source data, efficiency summaries, and evidence and validation manifests. It excludes raw source datasets, target-label vaults, private SQLite indexes, CodeBERT embeddings, feature caches, and per-function prediction or decision archives. The KBS submission source and PDF are supplied in the submission package; the GitHub link is the reproducibility artifact rather than a second authoritative copy of the manuscript. The extension-v2 configuration SHA-256 is `8AA39B0920D8CD2CFEFBF8C28109754F1B2DFA6049E17565122C6968E199AAD2`; independent artifact and evidence-validation reports are stored with the public snapshot.

Ethics statement

The study uses public source-code records and vulnerability labels for defensive classification. It does not generate exploits, access live systems, or involve human participants; institutional ethics review is not applicable subject to institutional policy.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this manuscript, the authors used OpenAI Codex, a generative-AI tool, to assist with research-question exploration,

methodological planning, code drafting and debugging, analysis interpretation, and manuscript drafting. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. No generative AI was used to fabricate data, results, or figures.

CRediT authorship contribution statement

Yanbo Bian: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Software, Validation, Visualization, Writing–original draft, Writing–review and editing. Zhihai Wang: Conceptualization, Formal analysis, Methodology, Supervision, Validation, Writing–review and editing.

References

[1] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, in: 2025 IEEE/ACM 47th International Conference on Software Engineering, 2025, pp. 1729–1741. Doi: 10.1109/ICSE55347.2025.00038.

[2] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, 2023, pp. 654–668. Doi: 10.1145/3607199.3607242.

[3] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, O. De Vel, P. Montague, Cross-project transfer representation learning for vulnerable function discovery, IEEE Transactions on Industrial Informatics 14 (2018) 3289–3297. Doi: 10.1109/TII.2018.2821768.

[4] S. Liu, G. Lin, L. Qu, J. Zhang, O. De Vel, P. Montague, Y. Xiang, CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Transactions on Dependable and Secure Computing 19 (2022) 438–451. Doi: 10.1109/TDSC.2020.2984505.

[5] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Transactions on Software Engineering 49 (2023) 4152–4168. Doi: 10.1109/TSE.2023.3285910.

[6] V. Nguyen, T. Le, C. Tantithamthavorn, J. Grundy, D. Phung, Deep domain adaptation with max-margin principle for cross-project imbalanced software vulnerability detection, ACM Transactions on Software Engineering and Methodology 33 (2024) 1–34. Doi: 10.1145/3664602.

[7] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, Journal of Systems and Software 213 (2024) 112038. Doi: 10.1016/j.jss.2024.112038.

[8] R. Haque, A. Ali, S. McClean, N. Khan, A zero-shot framework for cross-project vulnerability detection in source code, Empirical Software Engineering 31 (2026) 3. Doi: 10.1007/s10664-025-10749-4.

[9] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1536–1547. Doi: 10.18653/v1/2020.findings-emnlp.139.

[10] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, ACM, 2022, pp. 608–620. doi:10.1145/3524842.3528452.

[11] B. Steenhoek, H. Gao, W. Le, Dataflow analysis-inspired deep learning for efficient vulnerability detection, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. Doi: 10.1145/3597503.3623345.

[12] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning, 2017, pp. 1321–1330. PMLR 70.

[13] Y. Geifman, R. El-Yaniv, SelectiveNet: A deep neural network with an integrated reject option, in: Proceedings of the 36th International Conference on Machine Learning, 2019, pp. 2151–2159. PMLR 97.

[14] Z. Zhou, C. Sha, X. Peng, On calibration of pre-trained code models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. Doi: 10.1145/3597503.3639126.

[15] H. Wang, P. Lenihan, Z. Wang, Enhancing deployment-time predictive model robustness for code analysis and optimization, in: Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization, ACM, 2025, pp. 31–46. doi:10.1145/3696443.3708959.

[16] R. Rathnasuriya, W. Yang, When to answer and when to defer: A decision framework for reliable code predictions, arXiv preprint arXiv:2605.19369 (2026). ArXiv:2605.19369.

[17] R. Yumlembam, B. Issac, S. M. Jacob, Enhancing decision-making in windows PE malware classification during dataset shifts with uncertainty estimation, Knowledge-Based Systems 331 (2026) 114723. doi:10.1016/j.knosys.2025.114723.

[18] I. Kalouptsoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, A. Chatzigeorgiou, Software vulnerability prediction: A systematic mapping study, Information and Software Technology 164 (2023) 107303. doi:10.1016/j.infsof.2023.107303.

[19] C. Theisen, L. Williams, Better together: Comparing vulnerability prediction models, Information and Software Technology 119 (2020) 106204. doi:10.1016/j.infsof.2019.106204.

[20] C. Liu, X. Chen, X. Li, Y. Xue, Making vulnerability prediction more practical: Prediction, categorization, and localization, Information and Software Technology 171 (2024) 107458. doi:10.1016/j.infsof.2024.107458.

[21] A. Wijekoon, N. Wiratunga, A user-centred evaluation of DisCERN: Discovering counterfactuals for code vulnerability detection and correction, Knowledge-Based Systems 278 (2023) 110830. doi:10.1016/j.knosys.2023.110830.

[22] Q. Zhao, C. Huang, L. Dai, VULDEFF: Vulnerability detection method based on function fingerprints and code differences, Knowledge-Based Systems 260 (2023) 110139. doi:10.1016/j.knosys.2022.110139.

[23] S. L. Mirtaheri, A. Pugliese, V. Pascucci, Automated vulnerability score prediction through lightweight generative AI, Knowledge-Based Systems 329 (2025) 114406. doi:10.1016/j.knosys.2025.114406.

[24] Y. Li, S. Chen, W. Yang, Estimating predictive uncertainty under program data distribution shift, arXiv preprint arXiv:2107.10989 (2021). ArXiv:2107.10989.

[25] R. J. Tibshirani, R. F. Barber, E. J. Candès, A. Ramdas, Conformal prediction under covariate shift, in: Advances in Neural Information Processing Systems 32, 2019. NeurIPS 2019.

[26] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster, Conformal risk control, in: International Conference on Learning Representations, 2024. ArXiv:2208.02814; ICLR 2024.

[27] A. Farinhas, C. Zerva, D. Ulmer, A. F. T. Martins, Non-exchangeable conformal risk control, in: International Conference on Learning Representations, 2024. ICLR 2024.

[28] D. C. Almeida, J. Bravo, J. Bono, P. Bizarro, M. A. T. Figueiredo, High probability risk control under covariate shift, in: Proceedings of the Fourteenth Symposium on Conformal and Probabilistic Prediction with Applications, volume 266, 2025, pp. 133–152. PMLR 266.